

CS 188: Inteligência Artificial

Processos de Decisão de Markov



Instrutores: Peyrin Kao e Emma Pierson

Universidade da Califórnia, Berkeley

Resenha: Expectimax Search

- Resultados incertos controlados pelo acaso, não por um adversário
- Por que não saberíamos qual será o resultado de uma ação?
- Aleatoriedade explícita: rolar dados
- Oponentes imprevisíveis: os fantasmas respondem aleatoriamente
- Ações podem falhar: ao mover um robô, as rodas podem deslizar
- Os valores agora devem refletir resultados médios (expectimax), não resultados de pior caso (minimax)

Busca expectimax: calcular a pontuação média

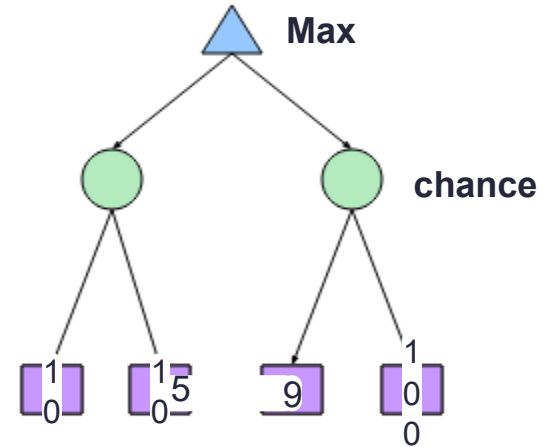
Jogo subótimo

Nós máximos como na busca minimax

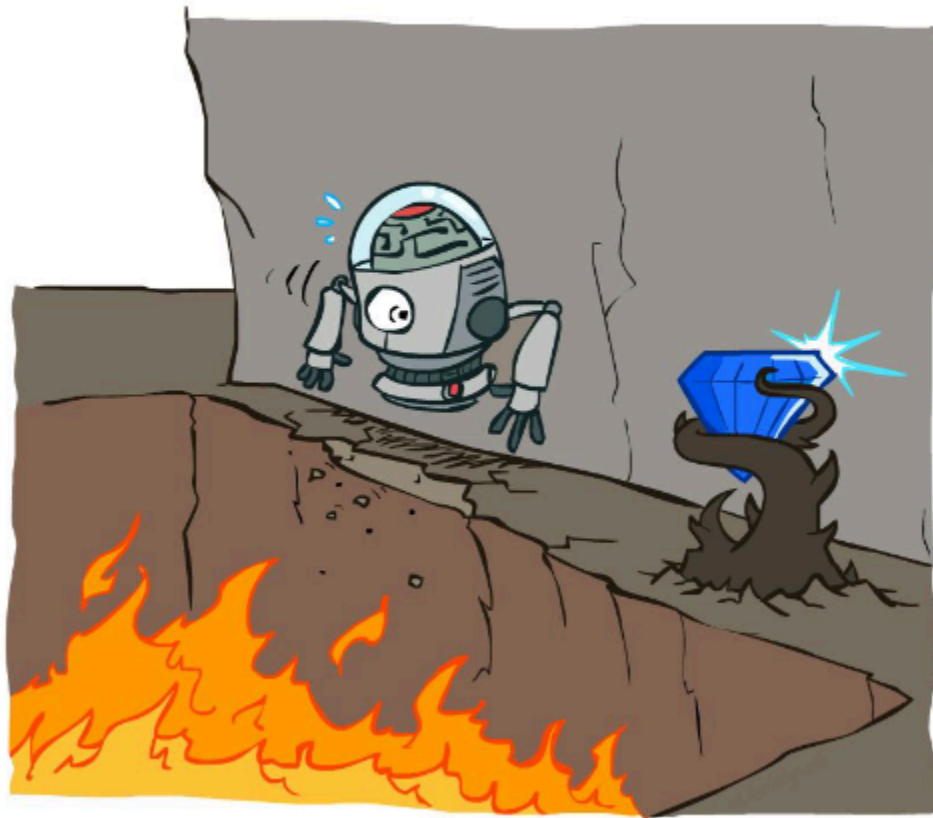
Nós de chance são como nós mínimos mas resultado é incerto

Calcule suas utilidades esperadas, ou seja, faça a média ponderada (expectativa) das crianças

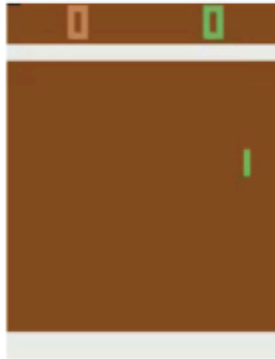
Hoje, aprenderemos como formalizar os problemas subjacentes de resultado incerto como Processos de Decisão de Markov



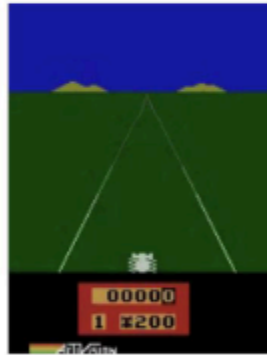
Busca Não Determinística



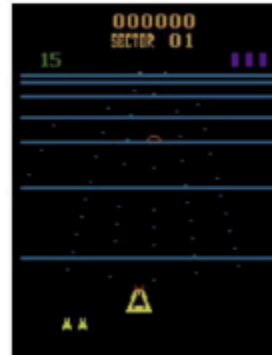
Aprendizado por reforço



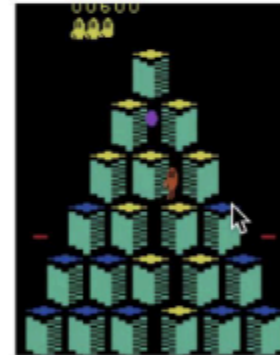
Pong



Enduro



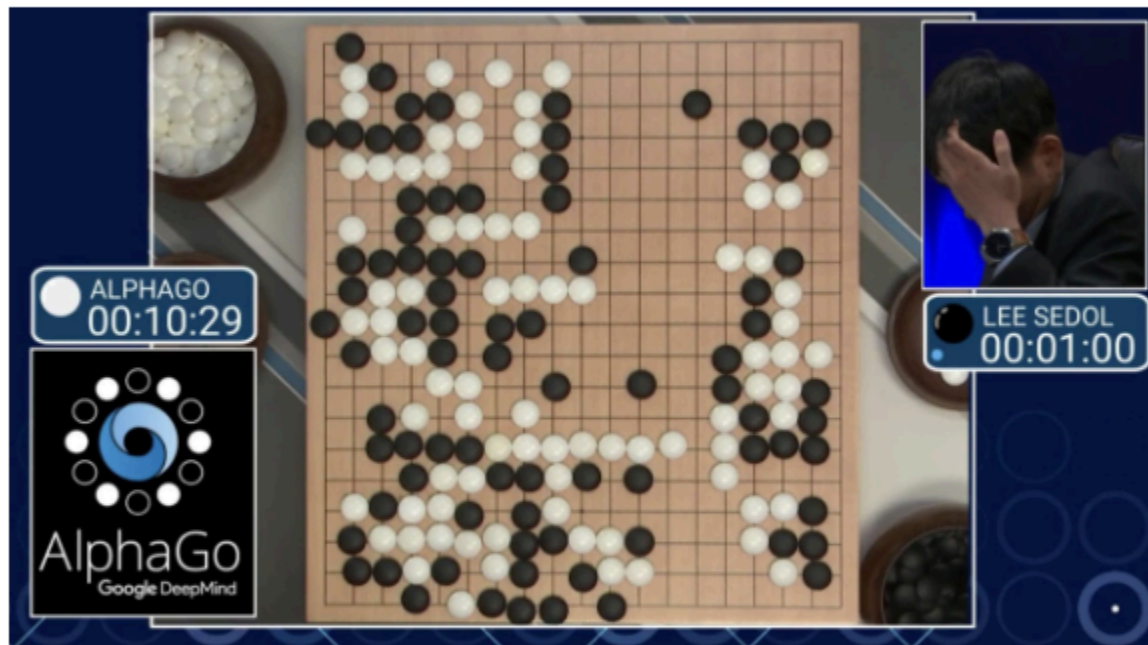
Beamrider



Q*bert

Atari (Minh et al, Nature 2015)

Aprendizado por reforço



AlphaGo Silver et al, Nature 2015

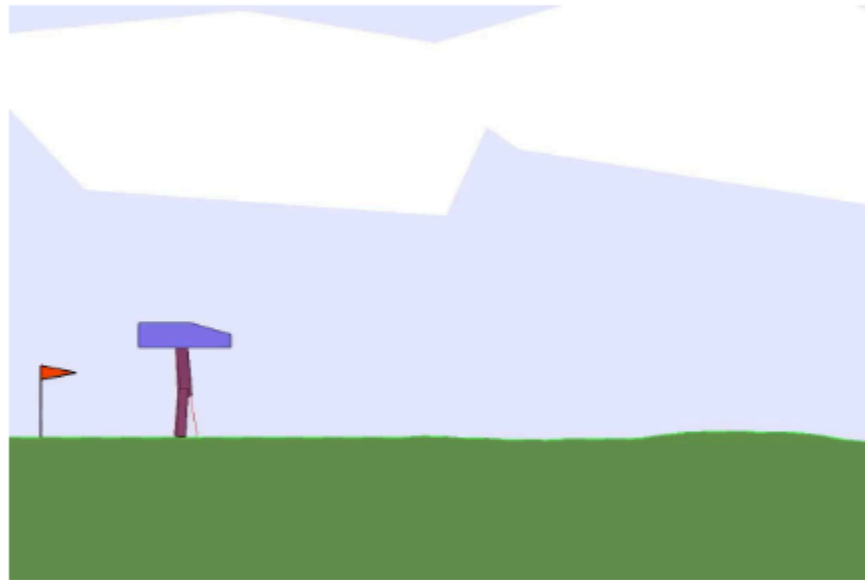
AlphaGoZero Silver et al, Nature 2017

AlphaZero Silver et al, 2017

Tian et al, 2016; Maddison et al, 2014; Clark et al, 2015

Aprendizado por reforço

Passo 0



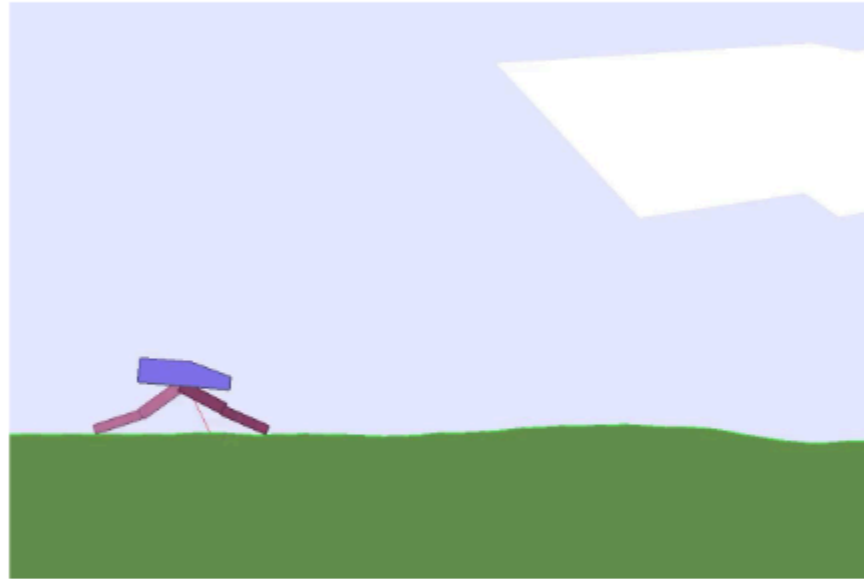
Schulman, Wolski, Dhariwal, Radford e Klimov, 2017

Schulman, Levine, Moritz, Jordan e Abbeel, 2015

Vídeo: Mukilan Krishnakumar

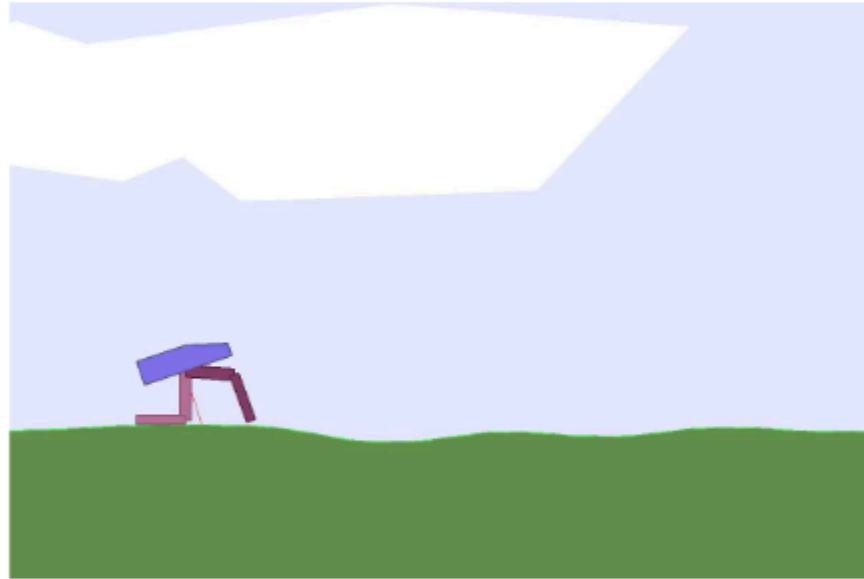
Aprendizado por reforço

Passo 100



Aprendizado por reforço

Passo 500

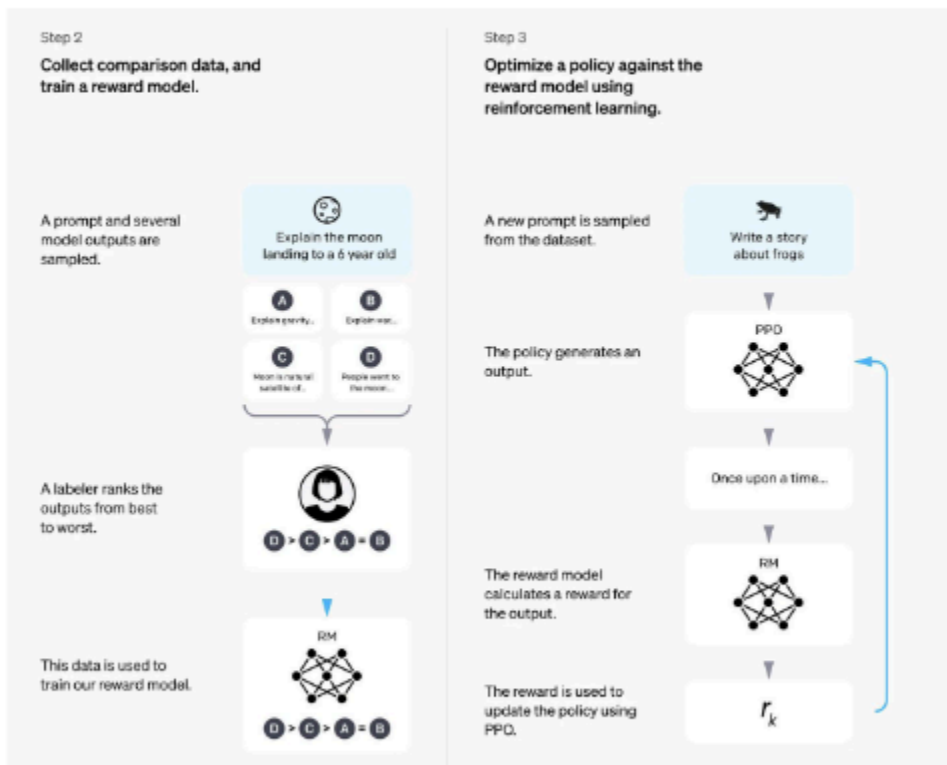


Aprendizado por reforço



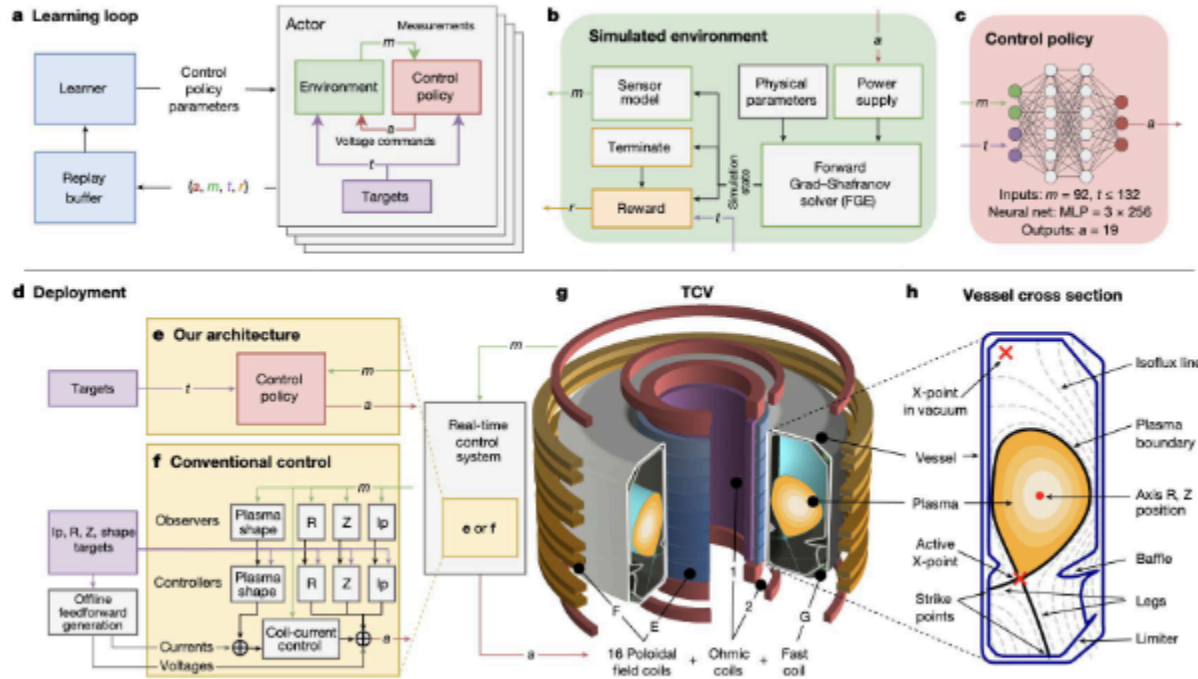
Inteligência Física, 2024

Aprendizado por reforço



OpenAI, 2022

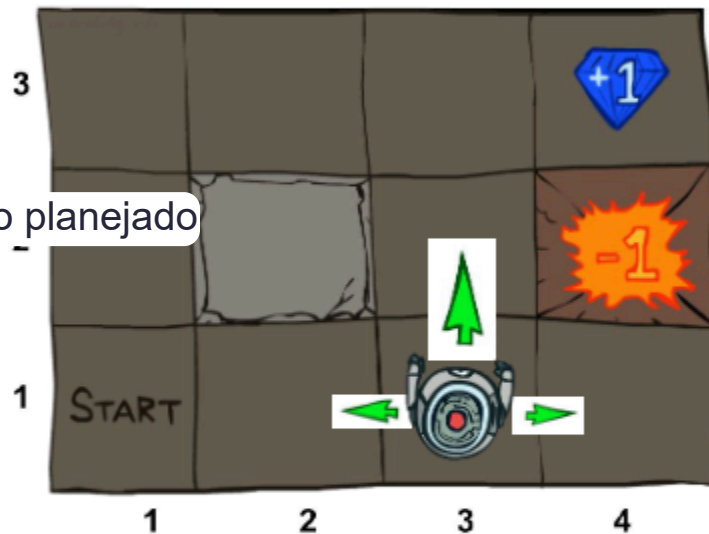
Aprendizado por reforço



DeGrave et al, Nature, 2022

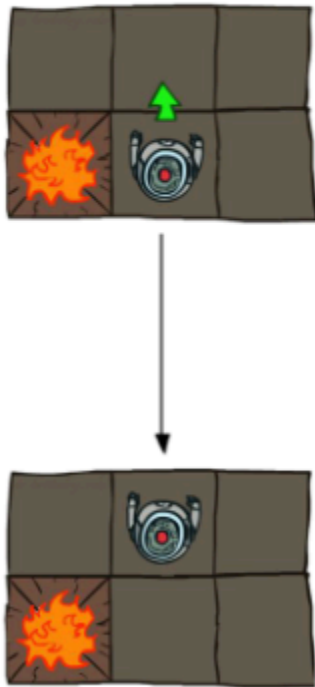
Exemplo: Grid World

- Um problema labiríntico
 - O agente vive em uma grade
 - Paredes bloqueiam o caminho do agente
- Movimento barulhento: as ações nem sempre saem como planejado
 - 80% das vezes, a ação Norte leva o agente para o Norte (se não houver parede ali);
 - 10% das vezes, Norte leva o agente para Oeste;
 - 10% Leste
 - Se houver um muro na direção em que o agente estaria Levado, o agente fica parado
- O agente recebe recompensas a cada passo de tempo
 - Pequenas recompensas "vivas" a cada etapa (podem ser negativas)
 - Grandes recompensas vêm no final (boas ou ruins)
- Objetivo: maximizar a soma das recompensas

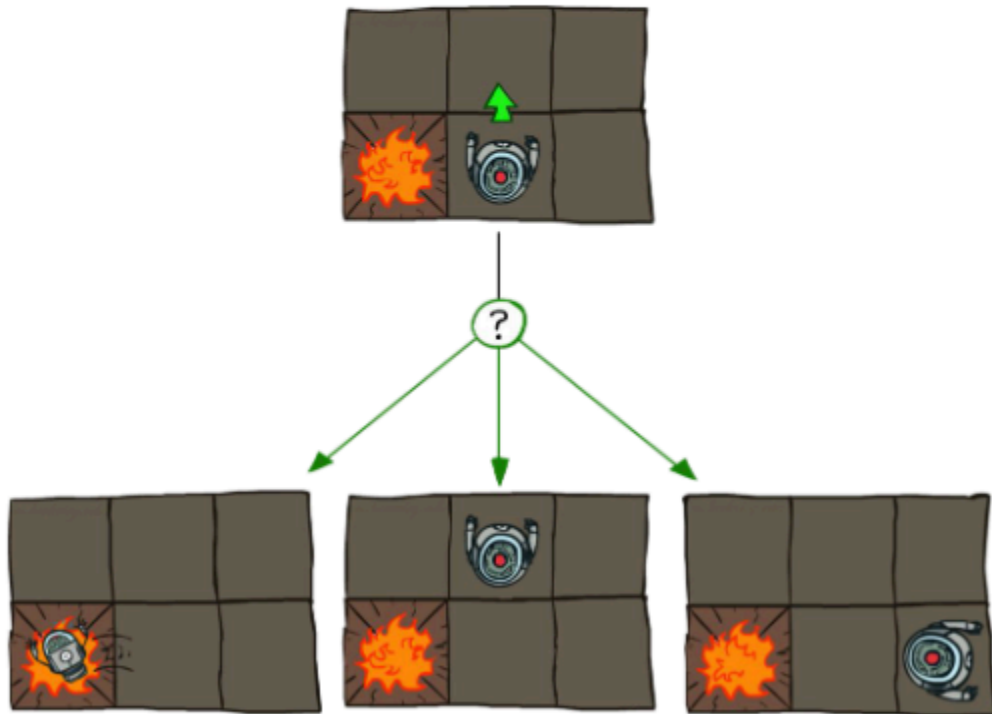


Ações do Mundo da Grade

Mundo de Grade Determinística

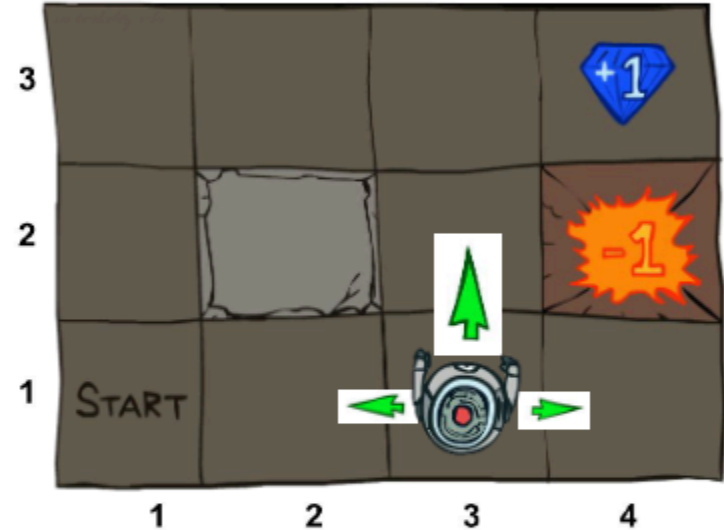


Mundo da Grade Estocástica



Processos de Decisão de Markov

- Um MDP é definido por:
 - Um conjunto de estados $s \in S$
 - Um conjunto de ações $a \in A$
 - Uma função de transição $T(s, a, s')$
 - Probabilidade de que a a de s leve a s', ou seja, $P(s'|s, a)$
 - Também chamado de modelo ou dinâmica
 - A função recompensa $R(s, a, s')$
 - Às vezes apenas $R(s)$ ou $R(s')$
- Um estado inicial Talvez um estado terminal
- MDPs sejam problemas de busca
- não determinística Uma forma de resolvê-los é com busca
- expectimax Teremos uma nova ferramenta em breve



Vídeo da Introdução do Manual de Demonstração do Gridworld



O que Markov tem sobre MDPs?

- "Markov" geralmente significa que, dado o estado atual, o Futuro e passado são independentes
- Para processos de decisão de Markov, "Markov" significa ação
Os resultados dependem apenas do estado atual

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0)$$

=

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

- Isso é como busca, onde a função sucessora só poderia
Depende do estado atual (não do histórico)



Andrey Markov
(1856-1922)

Políticas

- Em problemas determinísticos de busca de agente único, Queríamos um plano ótimo, ou sequência de ações, do início ao objetivo

- Para MDPs, queremos uma política ótima π^* : $S \rightarrow$

Um

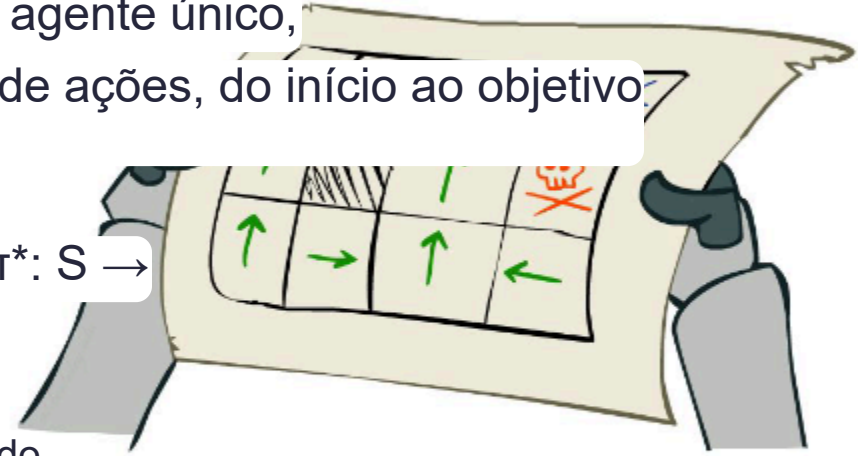
- Uma política π fornece uma ação para cada estado.

Uma política ótima é aquela que maximiza

utilitária se seguida. Uma política explícita define um agente reflexo

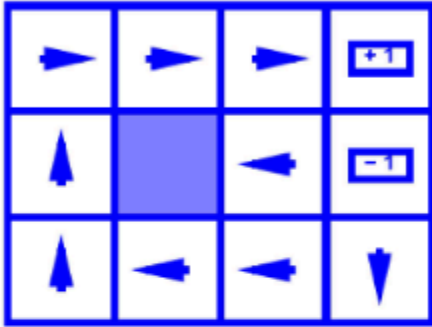
- Expectimax não calculou políticas inteiras
 - Ele calculava a ação apenas para um único estado

Esperado

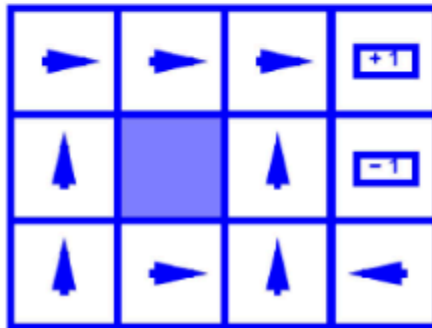
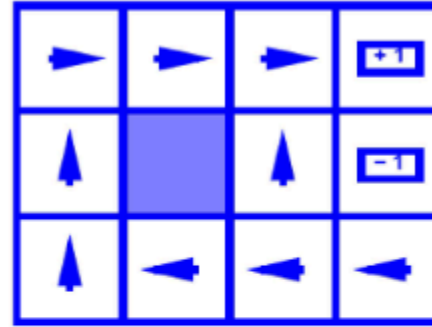


Política ótima quando $R(s, a, s') = -0,03$
para todos os não-terminais s

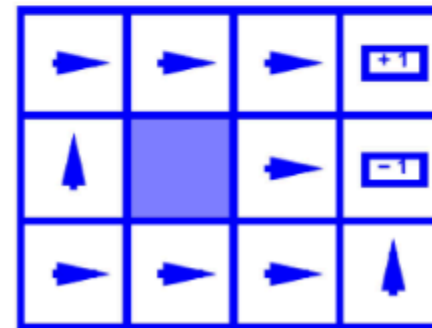
Políticas Ótimas



$$R(s) = -0,03 \quad R(s) = -0,01$$

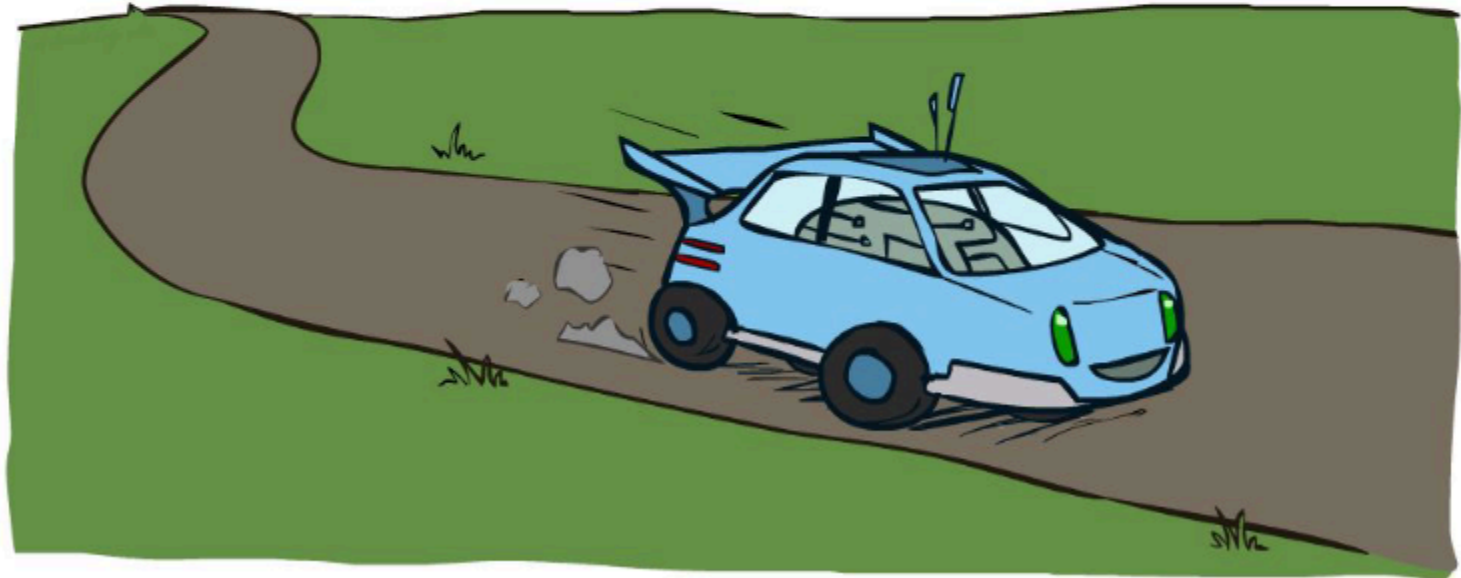


$$R(s) = -0,4$$



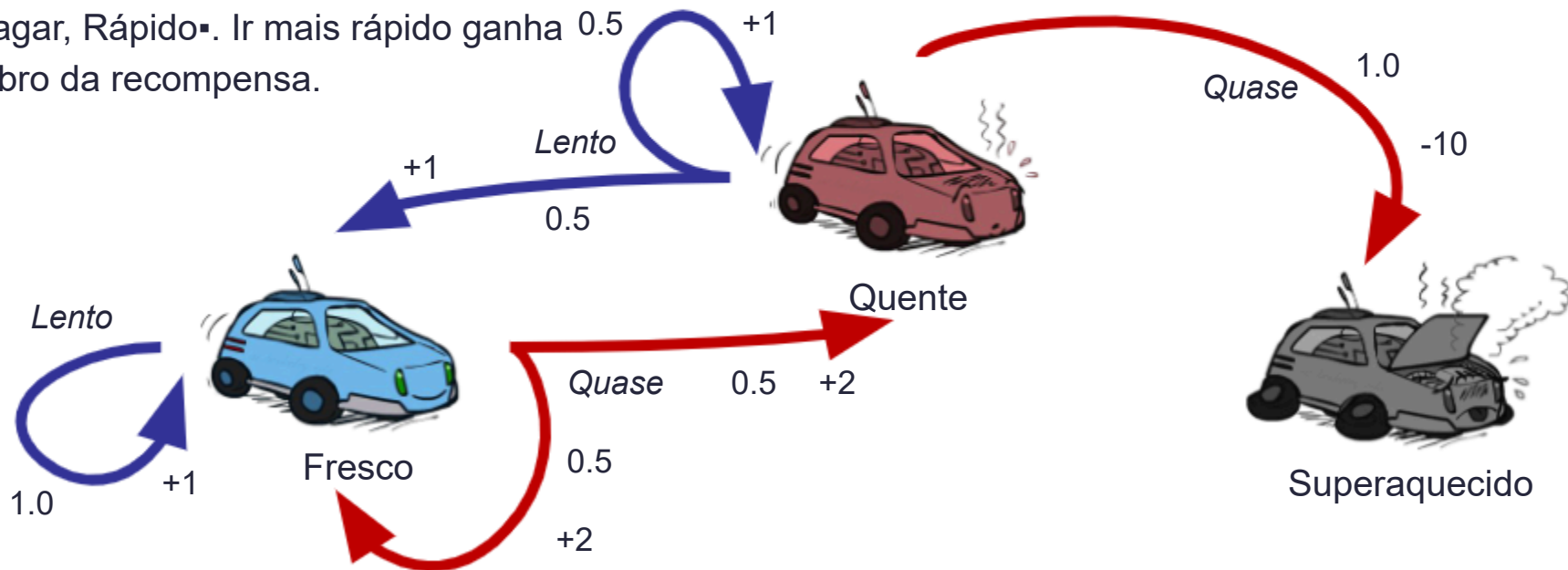
$$R(s) = -2,0$$

Exemplo: Corridas



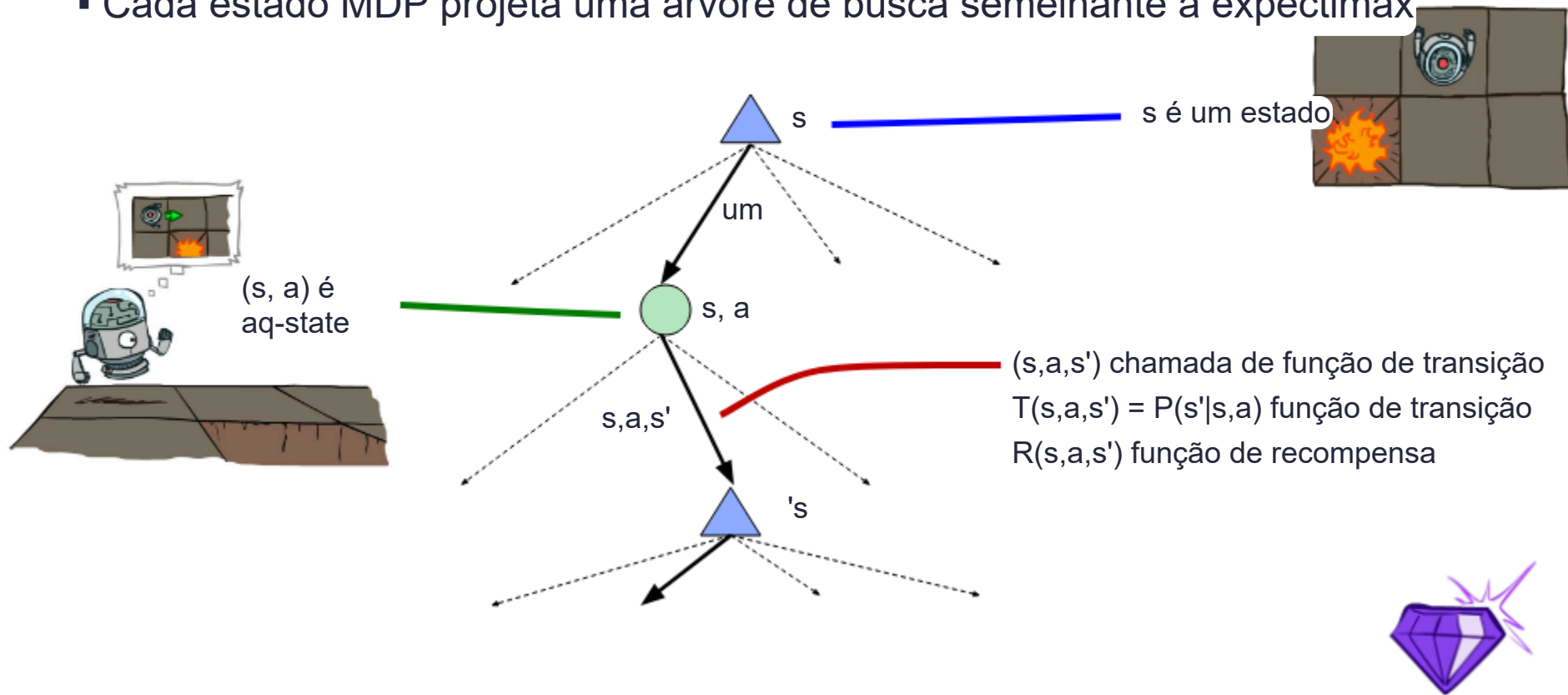
Exemplo: Corridas

- Um carro robô quer viajar longe, rapidamente▪. Três estados: Frio, Quente, Superaquecido▪. Duas ações: Devagar, Rápido▪. Ir mais rápido ganha 0.5 o dobro da recompensa.

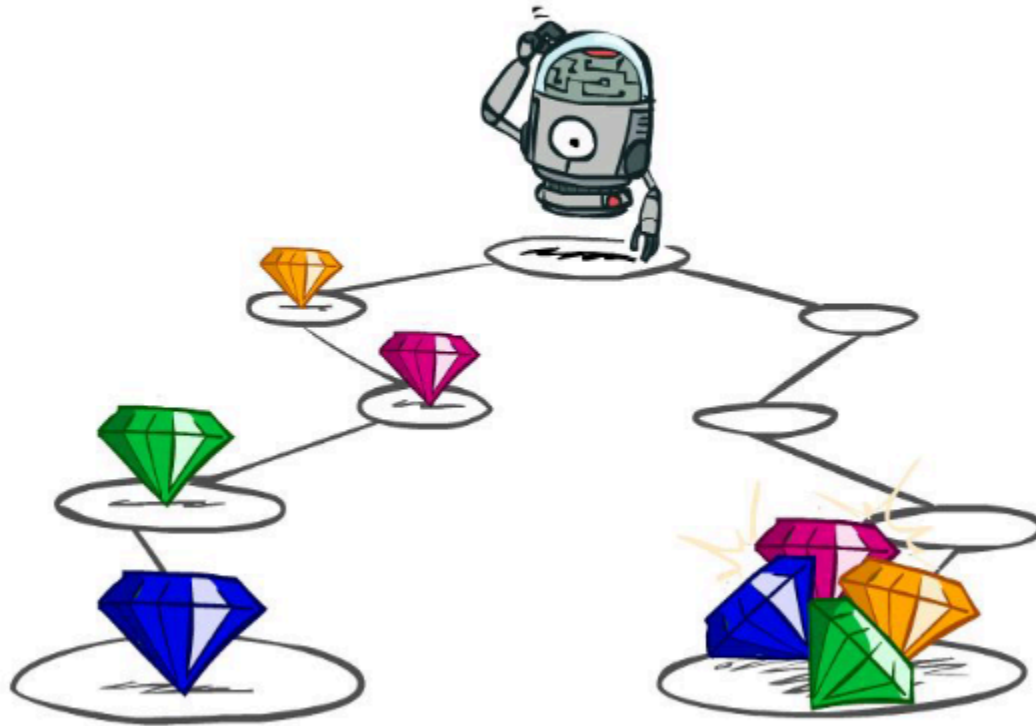


Árvores de Busca MDP

- Cada estado MDP projeta uma árvore de busca semelhante à expectimax

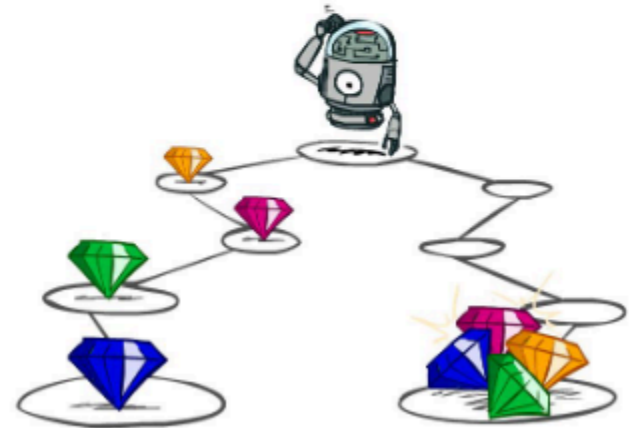


Utilidades de Sequências



Utilidades de Sequências

- Quais preferências um agente deve ter em relação às sequências de recompensa?
- Mais ou menos? $[1, 2, 2]$ $[2, 3, 4]$ ou
- Agora ou depois? $[0, 0, 1]$ $[1, 0, 0]$ ou



Descontando

- É razoável maximizar a soma das recompensas▪ Também é razoável preferir recompensas agora a recompensas depois▪ Uma solução: os valores das recompensas decaem exponencialmente



1

Vale a pena agora



γ

Vale a pena o próximo passo



γ^2

Vale a Pena em Dois Passos

Descontando

- Como fazer desconto?

- Cada vez que descemos um nível, nós multiplicamos o desconto uma vez



1

- Por que desconto?

- Recompensas mais cedo provavelmente têm utilidade maior do que recompensas posteriores
 - Também ajuda nossos algoritmos a convergir

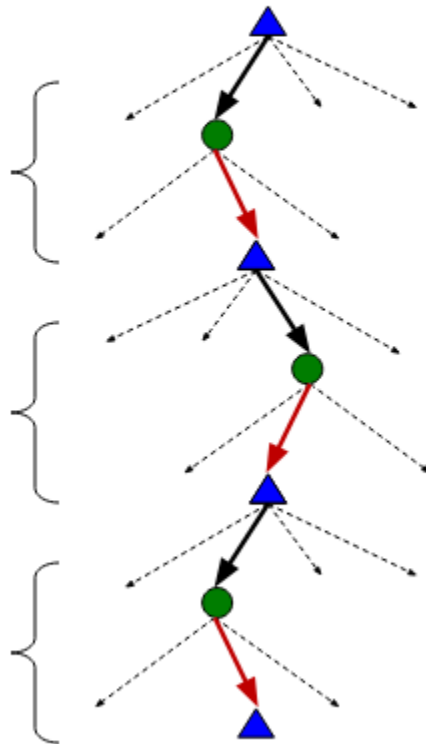
γ

- Exemplo: desconto de 0,5

- $U([1,2,3]) = 1*1 + 0,5*2 + 0,25*3$
 - $U([1,2,3]) < U([3,2,1])$



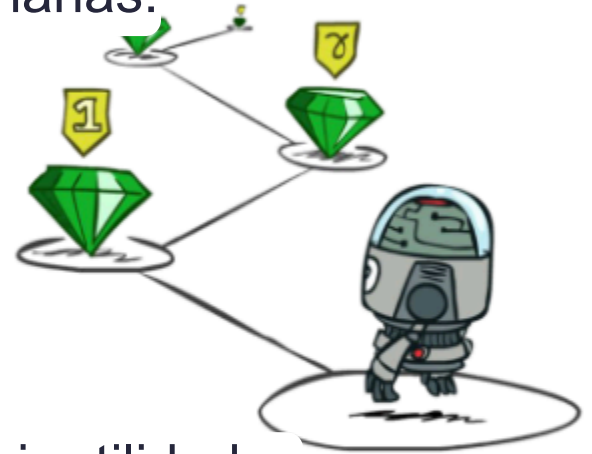
γ^2



Preferências de Estacionário

- Teorema: se assumirmos preferências estacionárias:

$$\begin{aligned} [a_1, a_2, \dots] &\succ [b_1, b_2, \dots] \\ \Leftrightarrow \\ [r, a_1, a_2, \dots] &\succ [r, b_1, b_2, \dots] \end{aligned}$$



- Então: existem apenas duas maneiras de definir utilidades
 - Utilidade aditiva: $([r_0, r_1, r_2, \dots]) = r_0 + r_1 + r_2 + \dots$
 - Utilidade descontada: $\sim ([r_0, r_1, r_2, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 \dots$

Quiz: Desconto

- Dado:

10				1
a	b	c	d	e

- Ações: Leste, Oeste e Saída (disponível apenas nos estados de saída a, e) ▪ Transições: determinísticas

- Quiz 1: Para $\gamma = 1$, qual é a política ótima? ▪ Quiz 2: Para $\gamma = 0,1$, qual é a política ótima? ▪ Quiz 3: Para quais γ Oeste e Leste são igualmente bons quando no estado d?

1

1

Utilidades Infinitas?!

- Problema: E se o jogo durar para sempre? Temos recompensas infinitas?
 - Por exemplo, jogo de corrida

- Soluções:

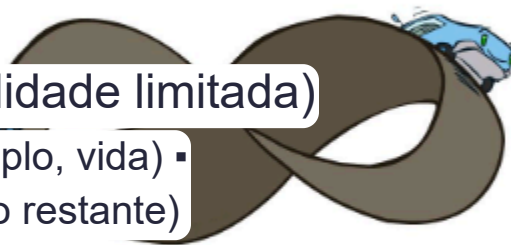
- Horizonte finito: (semelhante à busca com profundidade limitada)

- Encerrar episódios após um T de etapas fixas (por exemplo, vida)
 - Fornecer políticas não estacionárias (π depende do tempo restante)

- Desconto: use $0 < \gamma < 1$

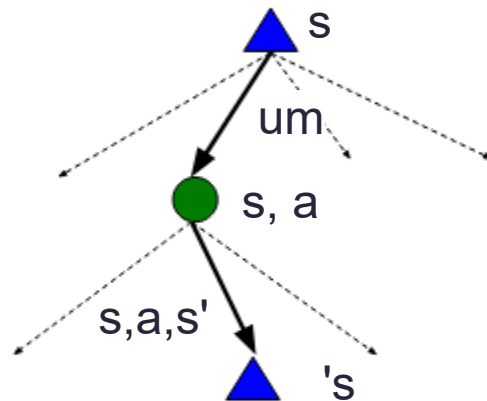
$$U([r_0, \dots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_{\max} / (1 - \gamma)$$

- Menor γ significa um "horizonte" menor – foco de curto prazo
 - Estado absorvente: garantir que, para toda apólice, um estado terminal eventualmente será ser alcançado (como "superaquecido" para corridas)



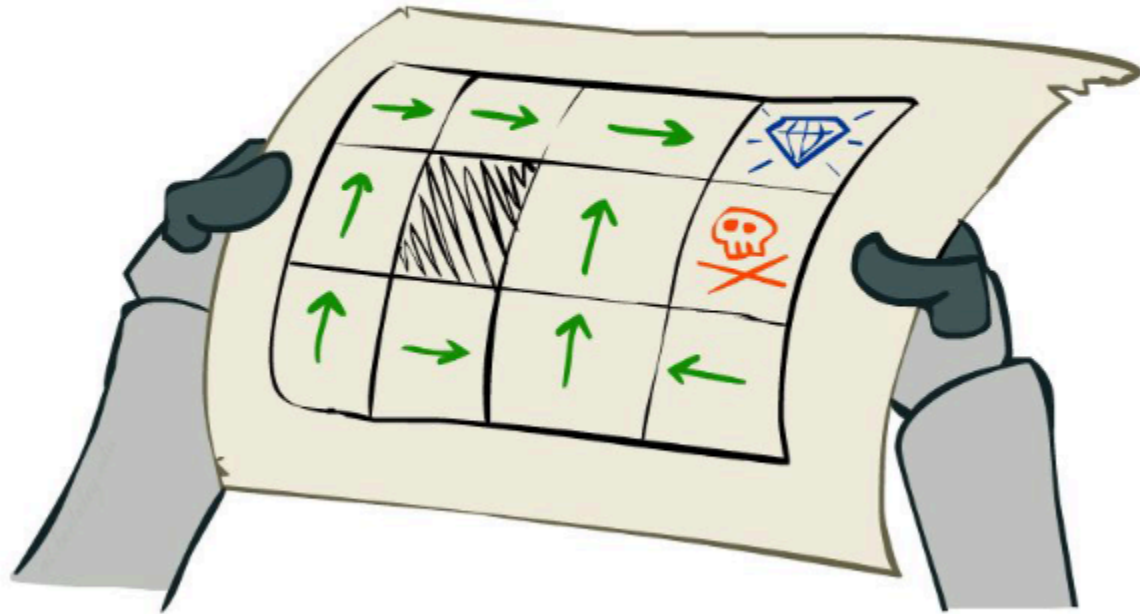
Resumo: Definindo MDPs

- Processos de decisão de Markov: ▪
- Conjunto de estados S ▪ Estado inicial s_0 ▪ Conjunto de ações A ▪
- Transições $P(s'|s,a)$ (ou $T(s,a,s')$) ▪
- Recompensa $R(s,a,s')$ (e desconto γ)



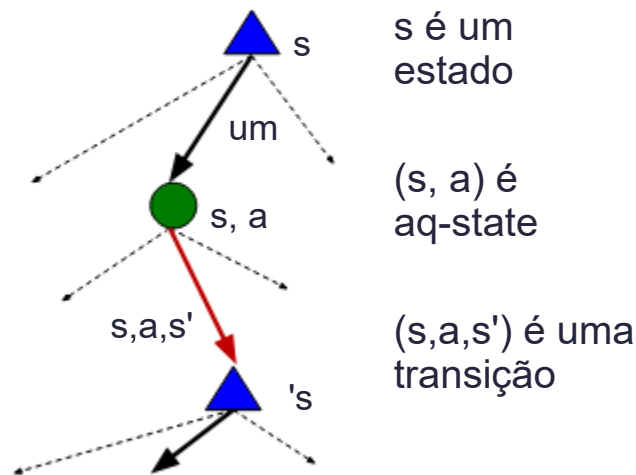
- Quantidades MDP até agora: ▪ Política = Escolha de ação para cada estado ▪ Utilidade = soma das recompensas (descontadas)

Resolvendo MDPs



Quantidades Ótimas

- O valor (utilidade) de um estado s : $V^*(s)$ = utilidade esperada começando em s e agindo de forma ótima
- O valor (utilidade) de um estado q (s,a) : $Q^*(s,a)$ = utilidade esperada começando a ter tomado a ação a a partir do estado s e (depois) agindo de forma ótima
- A política ótima: $\pi^*(s)$ = ação ótima a partir do estado s



Resumo da Demonstração – Valores do Gridworld V



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

Resumo da Demonstração – Valores Q do Gridworld



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

Valores dos Estados

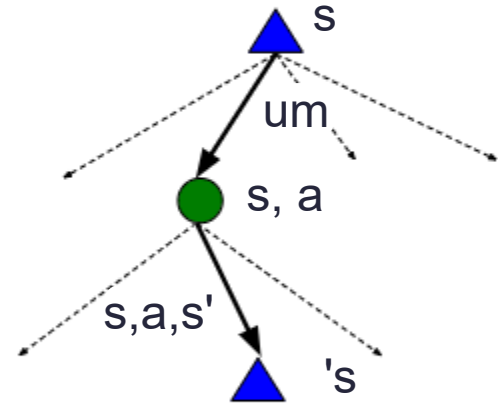
- Operação fundamental: calcular o valor (expectimax) de um estado
 - Utilidade esperada sob ação
 - ótima Soma média de recompensas
 - (descontadas) Isso é exatamente o que o expectimax calculou!

- Definição recursiva de valor:

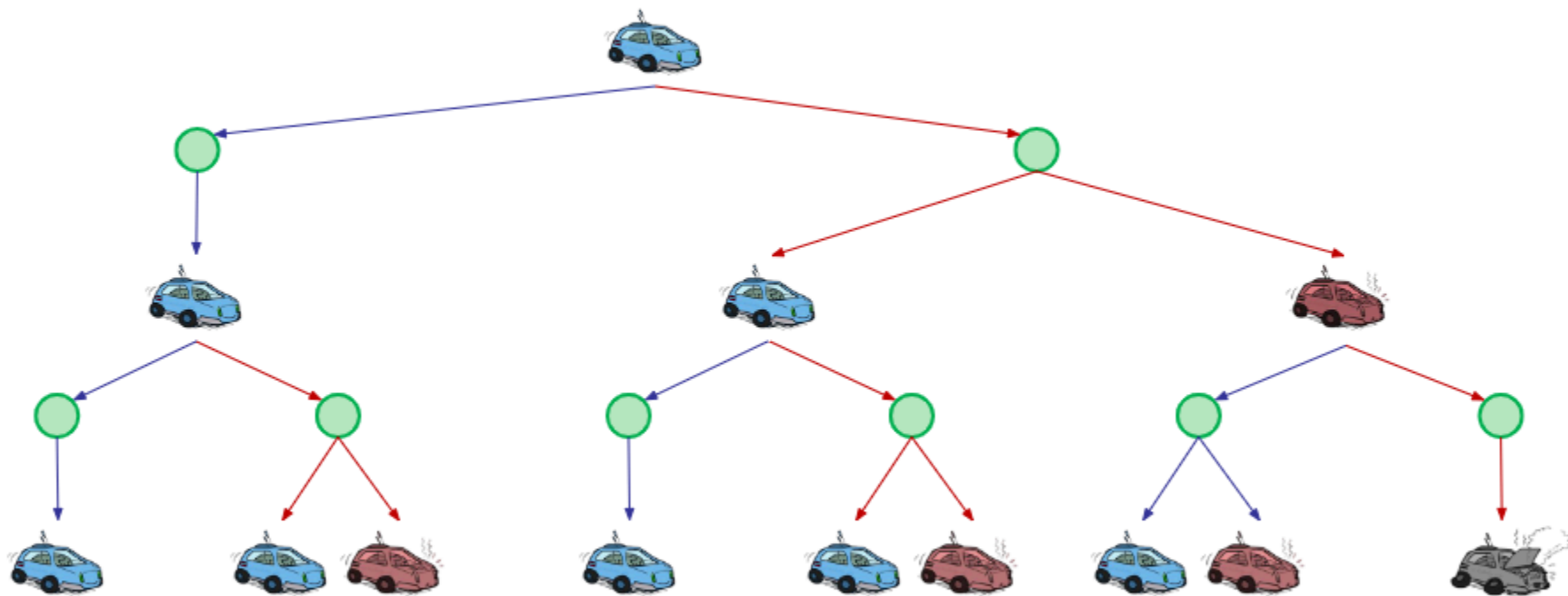
$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

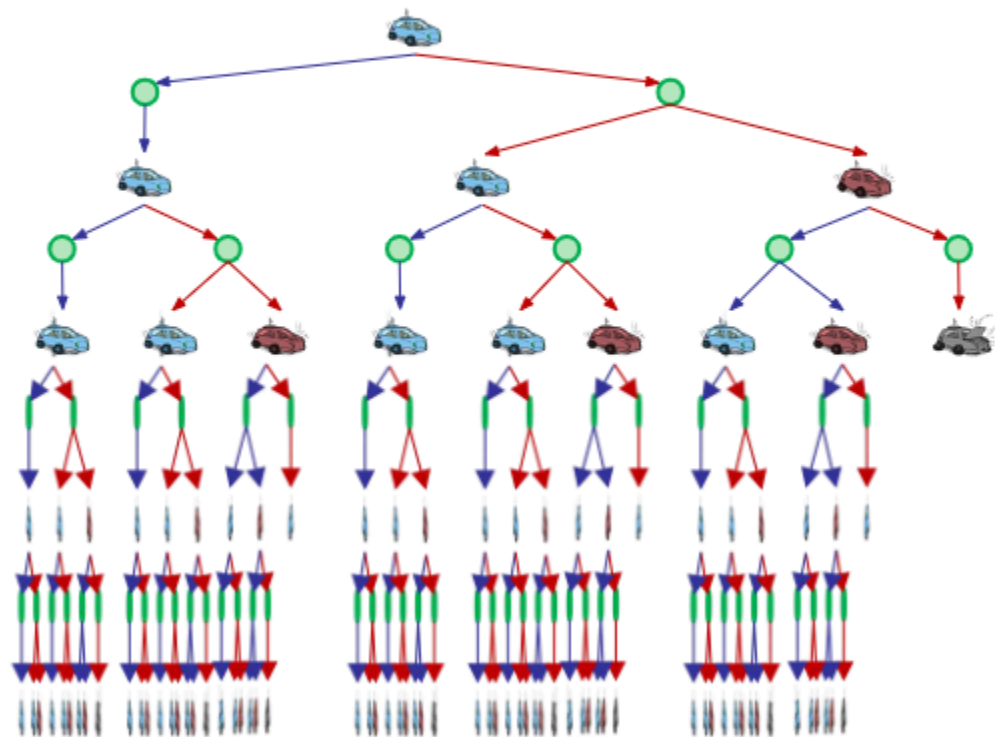
$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



Árvore de Busca de Corridas



Árvore de Busca de Corridas



Árvore de Busca de Corridas

- Estamos fazendo demais

Trabalhe com a Expectimax!

- Problema: Estados se repetem

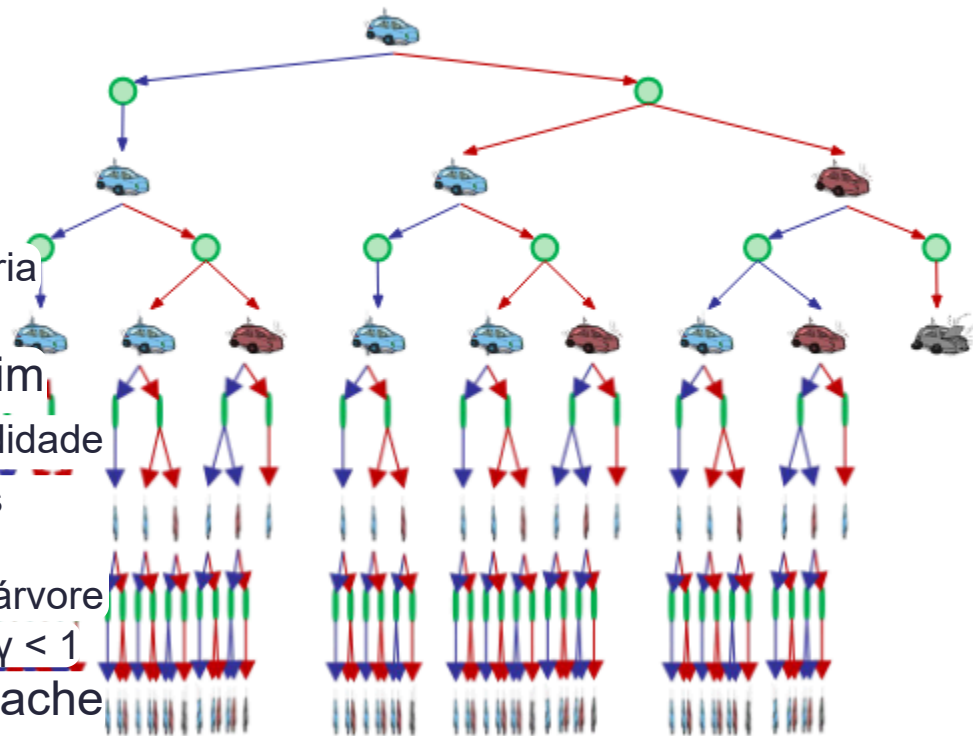
- Ideia: Apenas computação necessária
quantidades uma vez

- Problema: Árvore continua sem fim

- Ideia: Faça uma redução de profundidade computação, mas com profundidades crescentes até que a mudança seja pequena
 - Nota: partes profundas da árvore
- No fim das contas, não importa se $\gamma < 1$

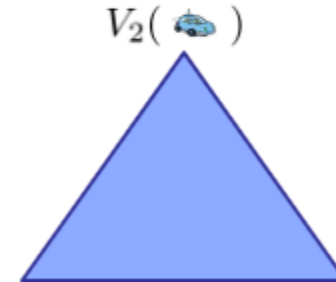
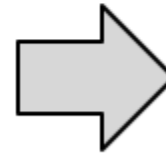
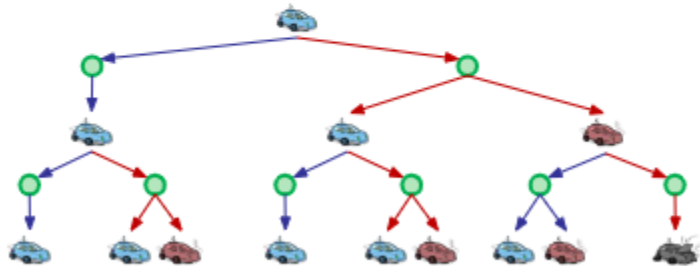
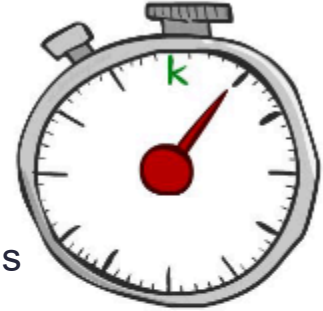
- Então talvez devêssemos fazer cache

E também ignorar partes muito profundas da árvore?

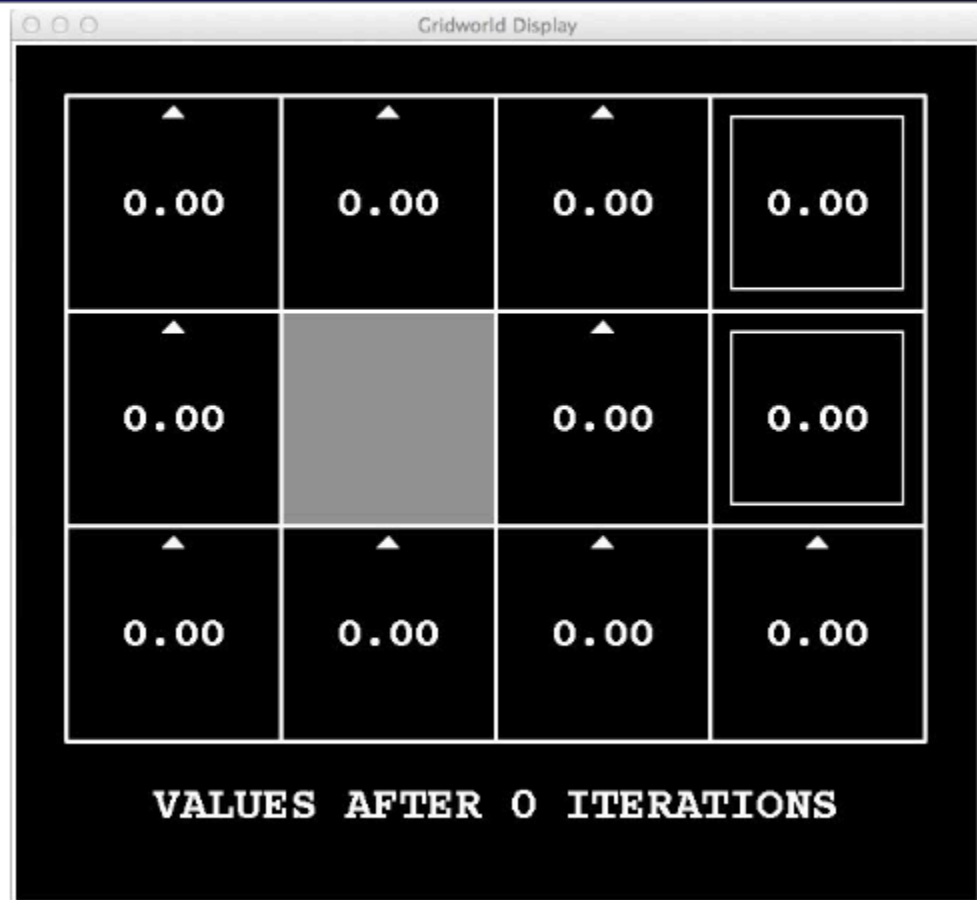


Valores com Limite de Tempo

- Ideia-chave: valores com limite de tempo Defina $V_k(s)$ como o valor ótimo de s se o jogo terminar em k mais passos de tempo
- Equivalentemente, é o que um expectimax de profundidade k daria de s



k=0



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=1



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=2



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=3



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=4



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=5



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=6



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=7



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=8



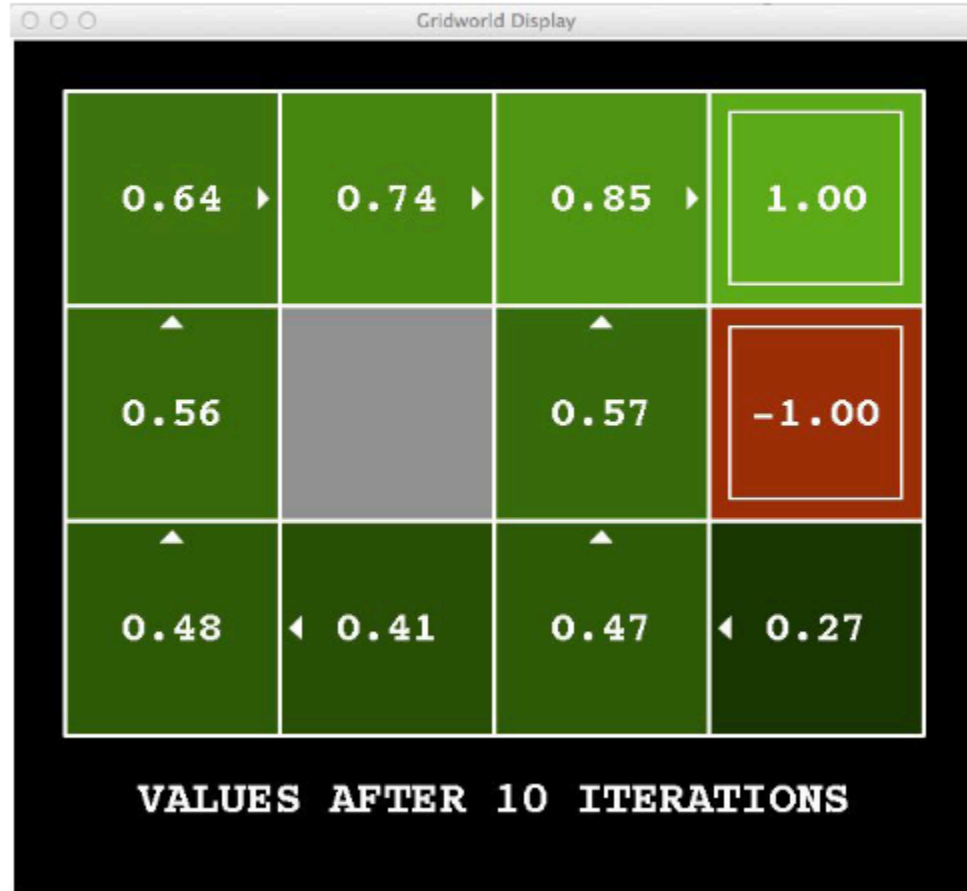
Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=9



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=10



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=11



Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=12



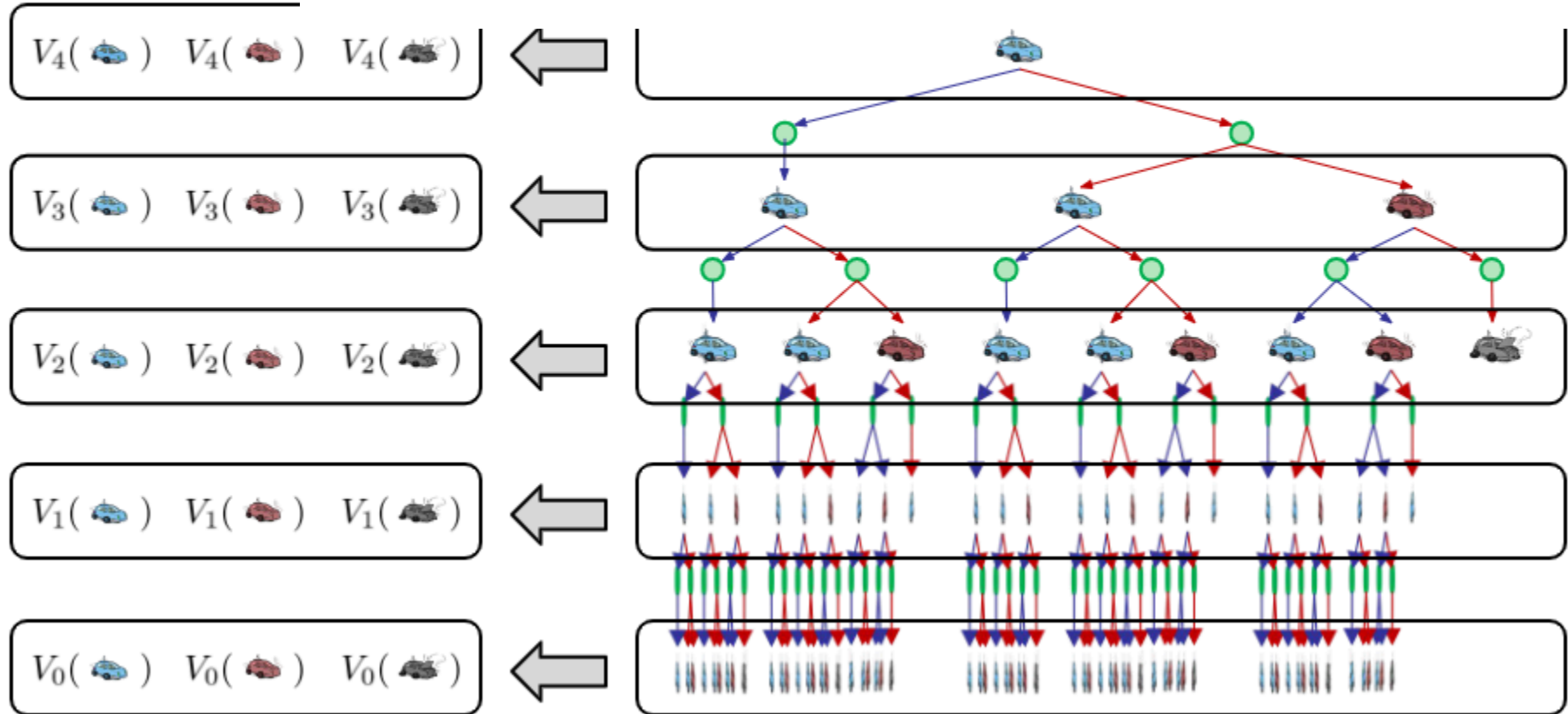
Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

k=100

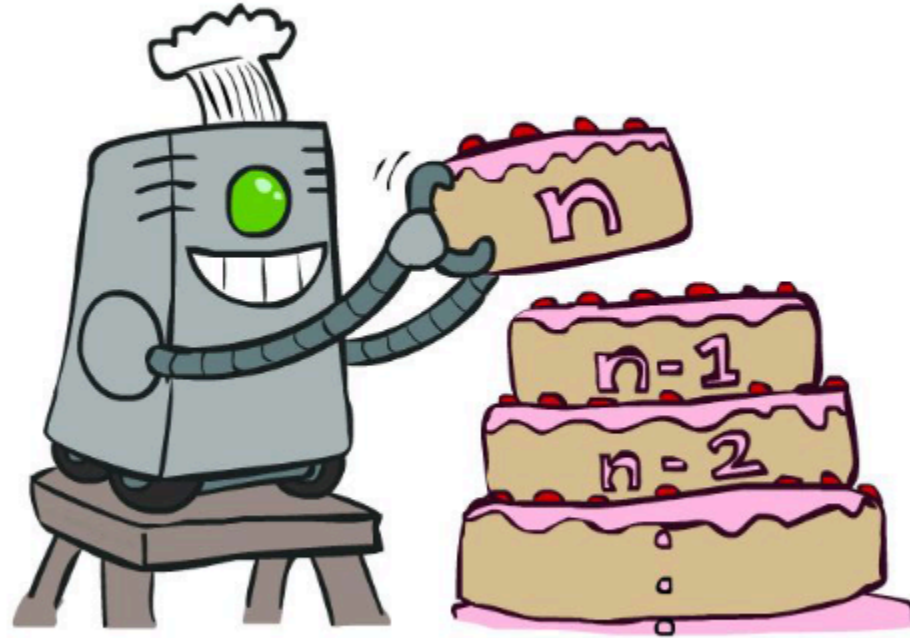


Ruído = 0,2
Desconto = 0,9
Recompensa de vida = 0

Cálculo de Valores Limitados de Tempo



Iteração de Valor

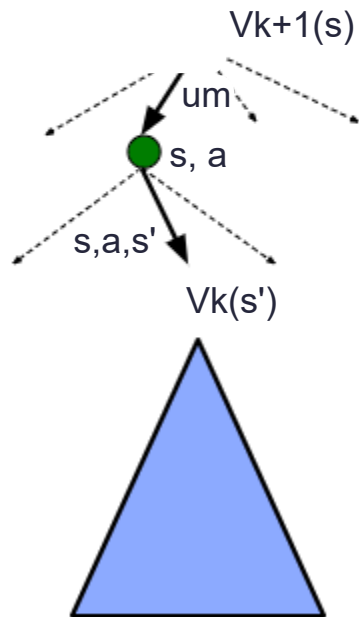


Iteração de Valor

- Comece com $V_0(s) = 0$: nenhum passo de tempo restante significa uma soma esperada de recompensa de zero
- Dado vetor de valores $V_k(s)$, faça uma linha de expectimax de cada estado:

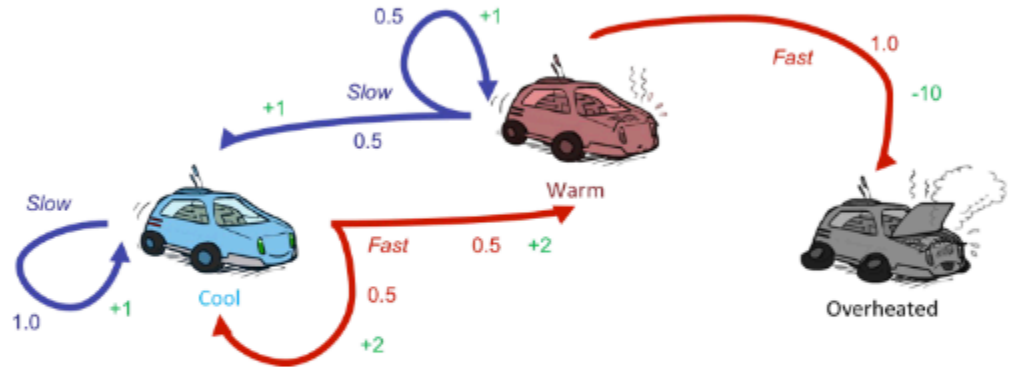
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

- Repita até a convergência
- Complexidade de cada iteração: Teorema $O(S^2A)$:
irá convergir para valores ótimos únicos
Ideia básica: as aproximações são refinadas para valores ótimos
A política pode convergir muito antes dos valores



Exemplo: Iteração de Valor

			
V_2	3.5	2.50	
V_1	21		0
V_0	00		0



Assuma que não há desconto!

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Na próxima entrevista: Métodos Baseados em Políticas
